

- [Python PEP8: Escribiendo Código Fácil de Leer](#)
 - [Introducción](#)
 - [Formatear Código Python PEP8: Linters y Autoformatters](#)
 - [Organización del código](#)
 - [Líneas en Blanco](#)
 - [Espacios en Blanco](#)
 - [Indentación del código](#)
 - [Tamaño de línea](#)
 - [Operaciones largas](#)
 - [Codificación de ficheros](#)
 - [Convenciones al Nombrar Elementos: CamelCase y snake_case](#)
 - [Eligiendo Nombres](#)
 - [Estilos: Camel Case y snake_case](#)
 - [Importando Paquetes: Orden y Organización](#)
 - [Comas al Final de Línea](#)
 - [Comentarios](#)

Python PEP8: Escribiendo Código Fácil de Leer

Introducción

La [PEP8](#) es una guía que indica las **convenciones estilísticas** a seguir para escribir código Python. Se trata de un conjunto de recomendaciones cuyo objetivo es ayudar a escribir código más legible y abarca desde cómo nombrar variables, al número máximo de caracteres que una línea debe tener.

De acuerdo con Guido van Rossum, **el código es leído más veces que escrito**, por lo que resulta importante escribir código que no sólo funcione correctamente, sino que además pueda ser leído con facilidad. Esto es precisamente lo que veremos en este artículo.

Code is read much more often than it is written, *Guido van Rossum*

Dos mismos códigos pueden realizar lo mismo funcionalmente, pero si no se siguen unas directrices estilísticas, se puede acabar teniendo un código muy difícil de leer.

Los problemas más frecuentes suelen ser:

- Líneas demasiado largas.
- Nombres de variables poco explicativos.
- Código mal comentado.
- Uso incorrecto de espacios y líneas en blanco.
- Código mal indentado.

Aunque es cierto que ciertas directrices pueden resultar arbitrarias, Python define en la PEP8 las normas estilísticas a seguir para cualquier código parte de la librería estándar, por lo que queda al criterio de cada uno usar estas recomendaciones o no. Sin embargo, prácticamente cualquier código o librería usado por gran cantidad de personas, emplea estas recomendaciones, al haber un amplio consenso en la comunidad.

Formatear Código Python PEP8: Linters y Autoformatters

A veces puede resultar complicado acordarnos de todas y cada una de las normas de la PEP8, por lo que hay herramientas que nos ayudan a corregir automáticamente o indicarnos donde hay problemas en nuestro código. Hay dos tipos de herramientas:

- Los **linters** como [flake8](#) o [pycodestyle](#).
- Y los **autoformatters** como [black](#) y [autopep8](#).

Los *autoformatters* se limitan a indicarnos donde nuestro código no cumple con las normas, y en ciertos casos realiza las correcciones automáticamente. Por ejemplo, podemos instalar [autopep8](#) y se puede instalar de la siguiente manera:

```
$pip install autopep8
```

Y si lo usamos sobre un [script.py](#) intentará corregir los problemas.

```
$autopep8 script.py -v -i
```

Veamos un ejemplo. Para alguien recién iniciado en Python, tal vez el siguiente código parezca válido, sin embargo alguien que conozca la PEP8 podrá identificar varios problemas.

```
# script.py
def MiFuncionSuma(A, B, C, imprime = True):
    resultado=A+B+C
    if imprime != False:
        print(resultado)
    return resultado
a          = 4
variable_b = 5
var_c      = 10

MiFuncionSuma(a, variable_b, var_c)
```

Usando el comando anterior, se nos informará de todas las reglas que nuestro código no cumple, para que las podamos corregir. Es importante notar que existen reglas que pueden ser corregidas automáticamente, y otras que no.

- Todo lo relativo al uso de espacios o líneas en blanco, **puede ser corregido automáticamente** por autopep8.
- Sin embargo autopep8 nunca modificará el nombre de una variable, por lo que si incumplimos alguna norma en lo relativo a nombrar variables **debemos corregir de forma manual** las ocurrencias.

El código anterior **incumple las siguientes reglas** :

- **E251**: Uso incorrecto de espacios en `imprime = True`, debería ser `imprime=True`.
- **E225**: Los operadores como el `+` deben usar espacios, `A + B + C`.
- **E712**: Usar `if imprime` en vez de `if imprime != False`.
- **E305**: Después de la declaración de una función debemos dejar dos espacios en blanco.
- **E221**: No debemos usar tantos espacios al usar el operador `=` creando variables.
- También tenemos otros problemas relacionados con cómo nombrar a funciones y variables. Las funciones y variables deben ir en **snake case**. Lo veremos en detalle más adelante.

Teniendo en cuenta lo mencionado, podemos implementar las correcciones para tener un código que cumple con la PEP8.

```
# script.py
def mi_funcion_suma(a, b, c, imprime=True):
    resultado = a + b + a
    if imprime:
        print(resultado)
    return resultado

a = 4
variable_b = 5
var_c = 10

mi_funcion_suma(a, variable_b, var_c)
```

Visto ya un ejemplo concreto, a continuación veremos las normas más importantes introducidas en la PEP8.

Organización del código

Líneas en Blanco

El uso de líneas en blanco mejora notablemente la legibilidad. Mucho código seguido puede ser difícil de leer, pero un uso excesivo de líneas en blanco puede ser molesto. Python deja su uso a nuestro criterio, siempre y cuando cumplamos lo siguiente:

- **Rodear las funciones y clases con dos líneas en blanco** . Cada vez que definamos una clase o una función es necesario dejar dos líneas en blanco por arriba y dos por abajo.
- **Dejar una línea en blanco entre los métodos de una clase** . Los métodos de una clase deberán tener una línea en blanco entre ellos.
- **Usar líneas en blanco para agrupar pasos similares** . Si tenemos un conjunto de código que realiza una función concreta, es conveniente delimitarlo con una línea en blanco, de la misma manera que un libro separa ideas en párrafos.

```
# 1 espacio entre métodos
# 2 espacios entre clases y funciones
class ClaseA:
    def metodo_a(self):
        pass

    def metodo_b(self):
        pass
```

```
class ClaseB:
    def metodo_a(self):
        pass

    def metodo_b(self):
        pass

def funcion():
    pass
```

También resulta conveniente separar con una línea diferentes funcionalidades. La siguiente función calcula la [media](#) y la [mediana](#), por lo que las separamos con una línea en blanco.

```
def calcula_media_mediana(valores):
    # Calculamos la media
    suma_valores = 0
    for valor in valores:
        suma_valores += valor
    media = suma_valores / len(valores)

    # Calculamos la mediana
    valores_ordenados = sorted(valores)
    indice = len(valores) // 2
    if len(valores) % 2:
        mediana = valores_ordenados[indice]
    else:
        mediana = (valores_ordenados[indice]
                   + valores_ordenados[indice + 1]) / 2

    return media, mediana
```

Espacios en Blanco

El uso de espacios en blanco puede resultar clave para mejorar la legibilidad de nuestro código, y es por lo que la PEP8 nos dice **dónde debemos usar espacios y dónde no**. Se trata de buscar un punto de equilibrio entre un código demasiado disperso y con gran cantidad de espacios, y un código demasiado junto donde no se identifican sus partes.

Se nos recomienda **usar espacio** con operadores de [asignación](#).

```
# Correcto
x = 5

# Incorrecto
x=5
```

Y también con operadores [relacionales](#).

```
# Correcto
if x == 5:
    pass

# Incorrecto
if x==5:
    pass
```

Pero cuando tengamos funciones con argumentos por defecto, no debemos dejar espacios.

```
# Correcto
def mi_funcion(parameto_por_defecto=5):
    print(parameto_por_defecto)

# Incorrecto
def mi_funcion(parameto_por_defecto = 5):
    print(parameto_por_defecto)
```

Por otro lado se recomienda **no dejar espacios dentro del paréntesis** .

```
def duplica(a):
    return a * 2

# Correcto
duplica(2)

# Incorrecto
duplica( 2 )
```

Y tampoco entre **corchetes** .

```
# Correcto
lista = [1, 2, 3]
```

```
# Incorrecto
my_list = [ 1, 2, 3, ]
```

El uso de los espacios resulta muy útil cuando se combinan varios operadores utilizando diferentes variables, utilizando los espacios para agrupar **por orden de mayor prioridad** . Es por ello por lo que no dejamos espacios en `x**2` ni `(x-y)` dado que la potencia y el uso de paréntesis son los operadores con mayor prioridad.

```
# Correcto
y = x**2 + 1
z = (x-y) * (x+y)

# Incorrecto
y = x ** 2 + 5
z = (x - y) * (x + y)
```

Siguiendo la misma filosofía de agrupar por orden de ejecución, tenemos los siguiente ejemplos, siendo el primero el preferido por algunos *linters* .

```
# Correcto
if x > 0 and x % 2 == 0:
    print('...')

# Correcto
if x>0 and x%2==0:
    print('...')

# Incorrecto
if x% 2 == 0:
    print('...')
```

No usar espacio antes de `,` en llamadas a funciones o métodos.

```
# Correcto
print(x, y)

# Incorrecto
print(x , y)
```

Cuando usemos listas no usar espacios antes del índice o entre el índice y los `[]`.

```
# Correcto
lista[0]
```

```
# Incorrecto
lista [1]

# Incorrecto
lista [ 1 ]
```

Tampoco usando diccionarios.

```
# Correcto
diccionario['key'] = lista[indice]

# Incorrecto
diccionario ['key'] = lista [indice]
```

Por último y aunque pueda parecer raro para la gente que venga de otros lenguajes de programación, no se recomienda alinear las variables como se muestra a continuación.

```
# Correcto
var_a = 0
variable_b = 10
otra_variable_c = 3

# Incorrecto
var_a          = 0
variable_b     = 10
otra_variable_c = 3
```

Indentación del código

Como ya hemos visto en otros artículos, Python no usa `{ }` para designar bloques de código como otros lenguajes de programación, sino que usa bloques identados para indicar que un determinado bloque de código pertenece a por ejemplo un `if`.

```
if x > 5:
    pass
```

Un bloque identado **se representa usando cuatro espacios** y aunque el uso del tabulador pueda parecer lo mismo, Python 3 no recomienda su uso. Como regla de oro:

- Usa siempre cuatro espacios.
- Usa tabuladores si trabajas sobre código ajeno que ya use tabuladores.
- Bajo ningún concepto mezcles uso de espacios y tabuladores.

Por otro lado, también se puede indentar el código para evitar tener líneas muy largas, que resultan difíciles de leer. Es importante recordar que la PEP8 limita el tamaño de línea a 79 caracteres.

```
# Correcto
def mi_funcion(primer_parametro, segundo_parametro,
               tercer_parametro, cuarto_parametro,
               quinto_parametro):
    print("Python")

# Incorrecto
def mi_funcion(primer_parametro, segundo_parametro, tercer_parametro,
cuarto_parametro, quinto_parametro):
    print("Python")
```

Lo siguiente sería incorrecto ya que no se diferencian los argumentos de entrada del bloque de código a ejecutar por la función.

```
# Incorrecto
def mi_funcion(primer_parametro, segundo_parametro,
               tercer_parametro, cuarto_parametro,
               quinto_parametro):
    print("Python")
```

Análogamente se puede romper un `if` en diferentes líneas, útil cuando se usan gran cantidad de condiciones que no entran una una línea.

```
# Correcto
if (condicion_a and
    condicion_b):
    print("Python")
```

Tamaño de linea

Se recomienda limitar el tamaño de cada línea a **79 caracteres** , para evitar tener que hacer *scroll* a la derecha. Este límite también permite tener abiertos múltiples ficheros

en la misma pantalla, uno al lado de otro. Por otro lado se limita el uso de *docstrings* y comentarios a **72 caracteres** .

En los casos que tengamos una línea que no sea posible romper, podemos usar `\` para continuar con la línea en una nueva. Esto es algo que a veces puede darse en los [context managers](#).

```
# Correcto
with open('/esta/ruta/es/muy/pero/que/muy/larga/y/no/entra/en/una/sola/linea/') as
    fichero_1, \
        open('/esta/ruta/es/muy/pero/que/muy/larga/y/no/entra/en/una/sola/linea/',
            'w') as fichero_2:
    fichero_2.write(fichero_1.read())
```

Operaciones largas

Si queremos realizar una operación muy larga que no entra en una línea, tendremos que dividirla en múltiples. Lo recomendado es usar el operador al principio de cada línea, ya que resulta mas fácil de leer.

```
# Recomendado
income = (variable_a
          + variable_b
          + (variable_c - variable_d)
          - variable_e
          - variable_f)
```

La siguiente opción no es recomendada pero la PEP8 tampoco la prohíbe.

```
# No recomendado
income = (variable_a +
          variable_b +
          (variable_c - variable_d) -
          variable_e -
          variable_f)
```

Codificación de ficheros

Los ficheros se codifican por defecto en [ASCII](#) para Python 2 y [UTF-8](#) para Python 3, por lo que será necesario definir la codificación que usemos cuando queramos usar otro tipo.

Esto resulta muy importante, ya que si queremos almacenar una cadena que contiene caracteres no UTF-8 como **ó** y **ñ**, deberemos especificar el tipo de *encoding* de acuerdo a la [PEP263](#). El siguiente código puede dar problemas.

```
print("La acentuación del Español")  
# SyntaxError: Non-UTF-8 code starting with '\xf3' in file script.py on line 1, but  
# no encoding declared; see http://python.org/dev/peps/pep-0263/ for details
```

Sin embargo, con un pequeño cambio, podemos cambiar la forma en la que se codifica el texto.

```
# -*- coding: latin-1 -*-  
print("La acentuación del Español")  
# La acentuación del Español
```

Por otro lado, si tienes intención de desarrollar código para la librería estándar de Python, o contribuir en un proyecto con alcance global, debes saber lo siguiente:

- Todos los identificadores (como variables) deben usar ASCII.
- También deberán usar Inglés en la medida de lo posible, salvo abreviaciones.
- Las únicas excepciones son los test para código no ASCII y los nombres de autores.

Convenciones al Nombrar Elementos: CamelCase y snake_case

A la hora de escribir código, todo tiene nombres: variables, clases, funciones, paquetes, módulos, etc. Es por lo tanto muy importante seguir unas directrices determinadas para que nuestro código sea lo más legible posible. No se nombra igual a una clase que a una función, y tampoco suele ser recomendable usar nombres como **a** o **x** ya que aporta poca información. A continuación lo vemos en detalle.

Eligiendo Nombres

Antes de nada debemos pensar el nombre que le vamos dar a nuestra variable, clase o función. Es importante tener en cuenta lo siguiente:

- Evitar usar **palabras reservadas**. Si es necesario usar una palabra reservada como `class`, usar `class_` como alternativa.
- Evitar usar `l`, `O` y `I`, ya que pueden ser confundidas.
- Usar `_variable` para especificar uso interno. Por ejemplo `from m import *` no importaría lo que empieza con `_`.
- Se puede usar `__variable` para invocar el *name mangling* y hacer privadas determinadas variables o métodos.
- Para **métodos mágicos** usar siempre `__init__`, pero no son nombres que debemos crear sino reutilizar los que Python nos ofrece.

Estilos: Camel Case y snake_case

Supongamos que ya sabemos como vamos a nombrar a nuestra clase, función o variable. Pongamos que queremos llamar a nuestra función “mi función de prueba”. Dado que no podemos utilizar espacios para nombrar variables, hay diferentes alternativas:

- `mi_funcion_de_prueba`
- `MiFuncionDePrueba`
- `MIFUNCIONDEPRUEBA`
- `MI_FUNCION_DE_PRUEBA`
- `mifunciondeprueba`

Algunas de estas alternativas son conocidas como **Camel Case** o **snake_case** en el mundo de la programación. Pues bien, Python define cómo nombrar a cada tipo de la siguiente manera:

- **Funciones** : Letras en minúscula separadas por barra baja: `funcion`, `mi_funcion_de_prueba`.
- **Variables** : Al igual que las funciones: `variable`, `mi_variable`.
- **Clases** : Uso de CamelCase, usando mayúscula y sin barra baja: `MiClase`, `ClaseDePrueba`.
- **Métodos** : Al igual que las funciones, usar *snake case* : `metodo`, `mi_metodo`.

- **Constantes** : Nombrarlas usando mayúsculas y separadas por barra bajas: `UNA_CONSTANTE`, `OTRA_CONSTANTE`.
- **Módulos** : Igual que las funciones: `modulo.py`, `mi_modulo.py`.
- **Paquetes** : En minúsculas pero sin separar por barra bajas: `packete`, `mipaquete`

En el siguiente fragmento podemos ver su uso.

```
# mi_script.py
CONSTANTE_GLOBAL = 10

class MiClase():
    def mi_primer_metodo(self, variable_a, variable_b):
        return (variable_a + variable_b) / CONSTANTE_GLOBAL

mi_objeto = MiClase()
print(mi_objeto.mi_primer_metodo(5, 5))
```

Importando Paquetes: Orden y Organización

Los `import` deben separarse en diferentes líneas.

```
# Correcto
import os
import sys
```

```
# Incorrecto
import os, sys
```

Sin embargo cuando se importen varios elementos de una misma librería, si sería correcto importarlos en la misma línea.

```
# Correcto
from subprocess import Popen, PIPE
```

Con respecto a su **ubicación** , deberán seguir la siguiente:

- Deben ir al principio del fichero.
- Después de comentarios del módulo y *docstrings* .
- Antes de los `global` y las constantes.

Con respecto a su **organización** , debiendo haber una línea de separación entre cada grupo:

- Primero las librerías **estándar** .
- Segundo las librerías **externas** .
- Tercero las librerías **locales** .

Con respecto a su **tipo** :

- Se recomienda usar *imports absolutos* .
- Aunque también se permiten los **relativos** .

Por último, deben evitarse el `from <módulo> import *`. El uso de `*` importa todo lo presente en el `<módulo>`, por lo que no queda claro que se está usando y que no.

```
# Incorrecto
from collections import *
```

Si por ejemplo usamos únicamente `deque` y `defaultdict`, indicarlo.

```
# Correcto
from collections import deque, defaultdict
```

Comas al Final de Línea

El uso de comas al final de la línea suele ser opcional, salvo cuando se quiera crear **tuplas** de un sólo elemento como se muestra a continuación.

```
# Correcto
tupla = (1,)
print(tupla[0])
# Salida: 1
```

Sin embargo aunque su uso sea opcional en el resto de casos, en ciertas ocasiones puede estar justificado si por ejemplo tenemos una lista de elementos que puede cambiar con el tiempo. En este caso el uso de `,` al final puede ser de ayuda al sistema de control de versiones que utilicemos (como [Git](#)).

```
# Correcto
FICHEROS = [
    'fichero1.txt',
    'fichero2.txt',
]

# Incorrecto
FICHEROS = ['fichero1.txt', 'fichero2.txt',]
```

Comentarios

Los comentarios son muy importantes para realizar anotaciones a futuros lectores de nuestro código, y aunque resulta difícil definir cómo se debe comentar el código, hay ciertas directrices que debemos seguir:

- Cualquier comentario que contradiga el código es peor que ningún comentario. Por ello es muy importante que si actualizamos el código, no olvidarnos de actualizar los comentarios para evitar crear inconsistencias.
- Los comentarios deben ser frases completas, con la primera letra en mayúsculas.
- Si el comentario es corto, no hace falta usar el punto y final.
- Si el código es comentado en Inglés, usar [Strunk/White](#).
- Aunque cada uno es libre de escribir sus comentarios en el idioma que considere oportuno, se recomienda hacerlo en Inglés.
- Evitar comentarios poco descriptivos que no aporten nada más allá de lo que ya se ve a simple vista.
- En lo relativo a los comentarios *docstrings* , usar la [PEP257](#) como referencia.

A modo de ejemplo, como hemos explicado es conveniente evitar comentarios redundantes.

```
# Incorrecto
x = x + 1      # Suma 1 a la variable x
```

```
# Correcto
```

```
x = x + 1
```

```
# Compensa el offset producido por la medida
```